



Seguridad y Calidad en el Desarrollo (CDY2203)

Informe escrito

Ignacio Kríman Bravo

10/05/2026

Índice

Introducción	3
Vulnerabilidades OWASP 10 con ZAP	4
Análisis de Alertas	5
Reporte de Análisis Estático (SAST) y Composición de Software (SCA)	7
Reporte de Cobertura y Pruebas Unitarias (JaCoCo).....	8
Conclusión	10

Introducción

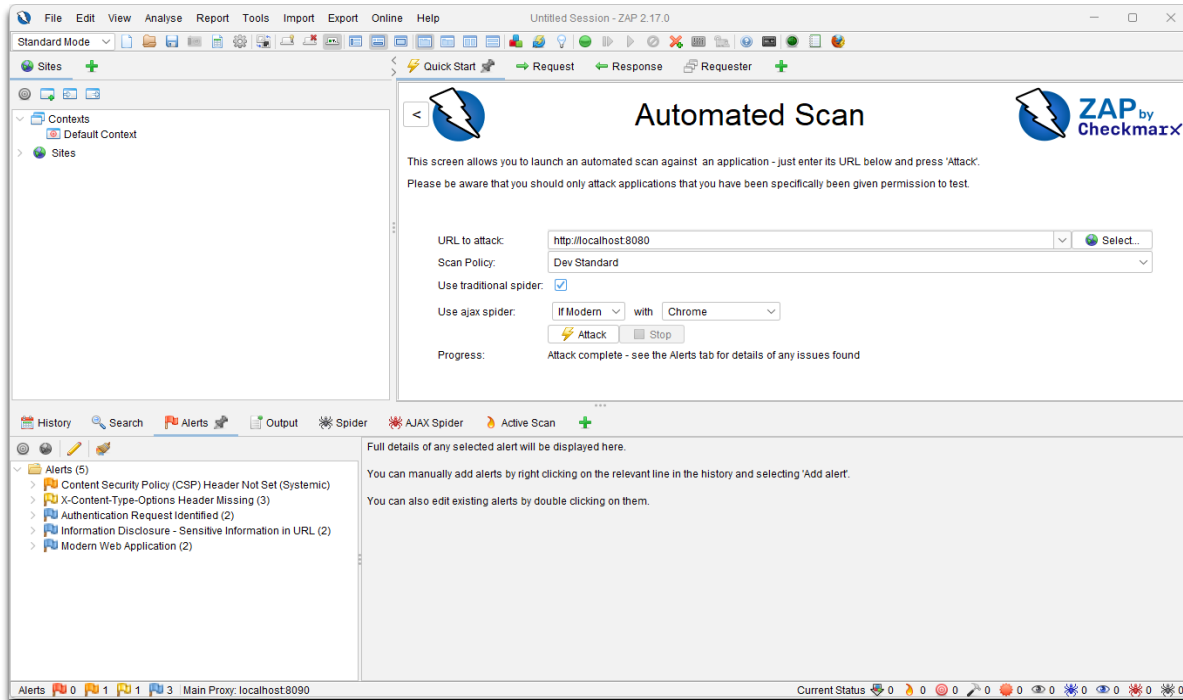
El presente informe detalla el proceso de auditoría, aseguramiento y control de calidad aplicado al sistema de gestión desarrollado en el ecosistema **Spring Boot 21**. En el contexto actual de la ciberseguridad, el desarrollo de software no solo debe enfocarse en la funcionalidad, sino también en la resiliencia ante ataques y la integridad de los datos.

Para este proyecto, se adoptó un enfoque de **Seguridad desde el Diseño (Security by Design)**, aplicando pruebas en diferentes etapas del ciclo de vida del desarrollo. El objetivo principal fue validar el cumplimiento de los estándares **OWASP Top 10**, asegurar que el código fuente esté libre de vulnerabilidades críticas mediante herramientas de análisis estático (**SAST**) y dinámico (**DAST**), y garantizar la estabilidad del sistema mediante una robusta suite de pruebas unitarias y de integración que cubren tanto la lógica de negocio como la capa de presentación (Frontend).

Vulnerabilidades OWASP 10 con ZAP

Se realizó un análisis dinámico sobre la aplicación en ejecución para identificar vulnerabilidades listadas en el estándar **OWASP Top 10**. Se ejecutó un **"Automated Scan"** incluyendo **Spidering** y **Active Scan**.

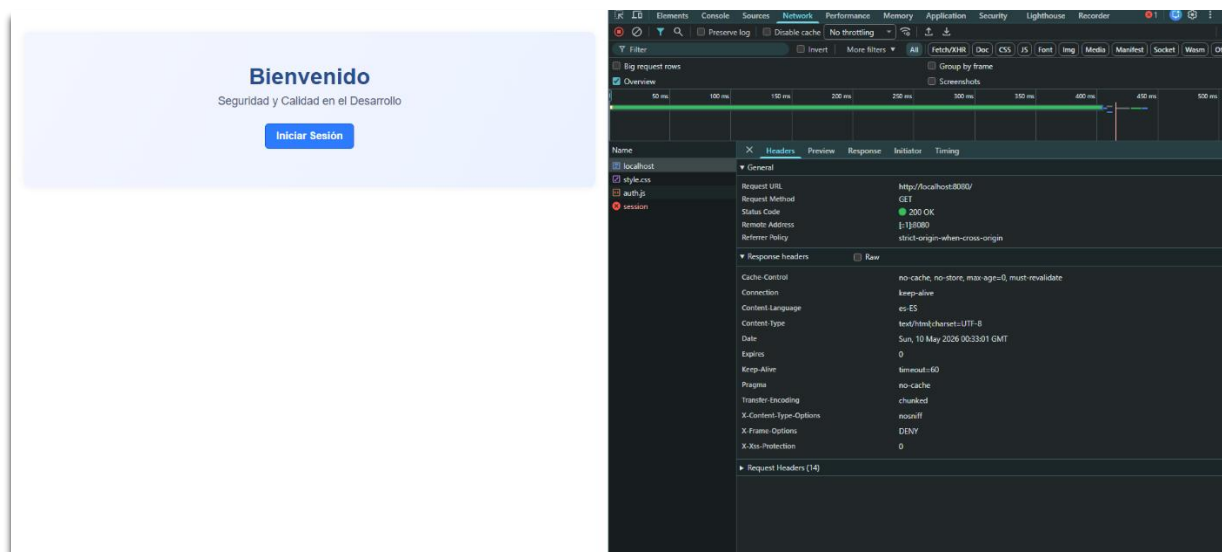
Se detectaron alertas de severidad media y baja, principalmente relacionadas con configuraciones de encabezados de seguridad (**CSP, X-Content-Type-Options**)



1. Resultados análisis ZAP

Análisis de Alertas

Alerta Detectada	Nivel de Riesgo	Explicación
Missing Anti-clickjacking Header	Medio	La aplicación no está enviando el encabezado X-Frame-Options , lo que permitiría que el sitio sea cargado dentro de un <iframe> en un sitio malicioso..
X-Content-Type-Options Header Missing	Bajo	El navegador podría intentar adivinar el tipo de contenido (MIME sniffing), ejecutando archivos como scripts cuando no deberían serlo. Esta es una configuración global. Se mitiga en la capa de seguridad de Spring , asegurando que el encabezado X-Content-Type-Options: nosniff esté presente en todas las respuestas de la API .
Cookie No HttpOnly Flag	Bajo	Indica que una cookie puede ser accedida mediante JavaScript , lo que facilita ataques XSS .



1- Resultado de los headers luego de la modificación de las alertas de ZAP.

Tras el análisis dinámico inicial realizado con **OWASP ZAP**, se identificaron vulnerabilidades de severidad "Media" y "Baja" relacionadas principalmente con la ausencia de políticas de seguridad en los encabezados **HTTP**. A continuación, se detallan las acciones de mitigación aplicadas en el código fuente.

1. Implementación de Content Security Policy (CSP)

- **Mitigación:** Se configuró en la capa de seguridad de **Spring** una política restrictiva que solo permite la carga de scripts y estilos desde fuentes confiables (self), minimizando la superficie de ejecución de scripts maliciosos.

2. Protección contra Clickjacking (X-Frame-Options)

- **Mitigación:** Se forzó el encabezado **X-Frame-Options** a **DENY** (o **SAMEORIGIN**), asegurando que la interfaz de la aplicación no pueda ser secuestrada visualmente mediante marcos externos.

3. Prevención de Mime-Sniffing (X-Content-Type-Options)

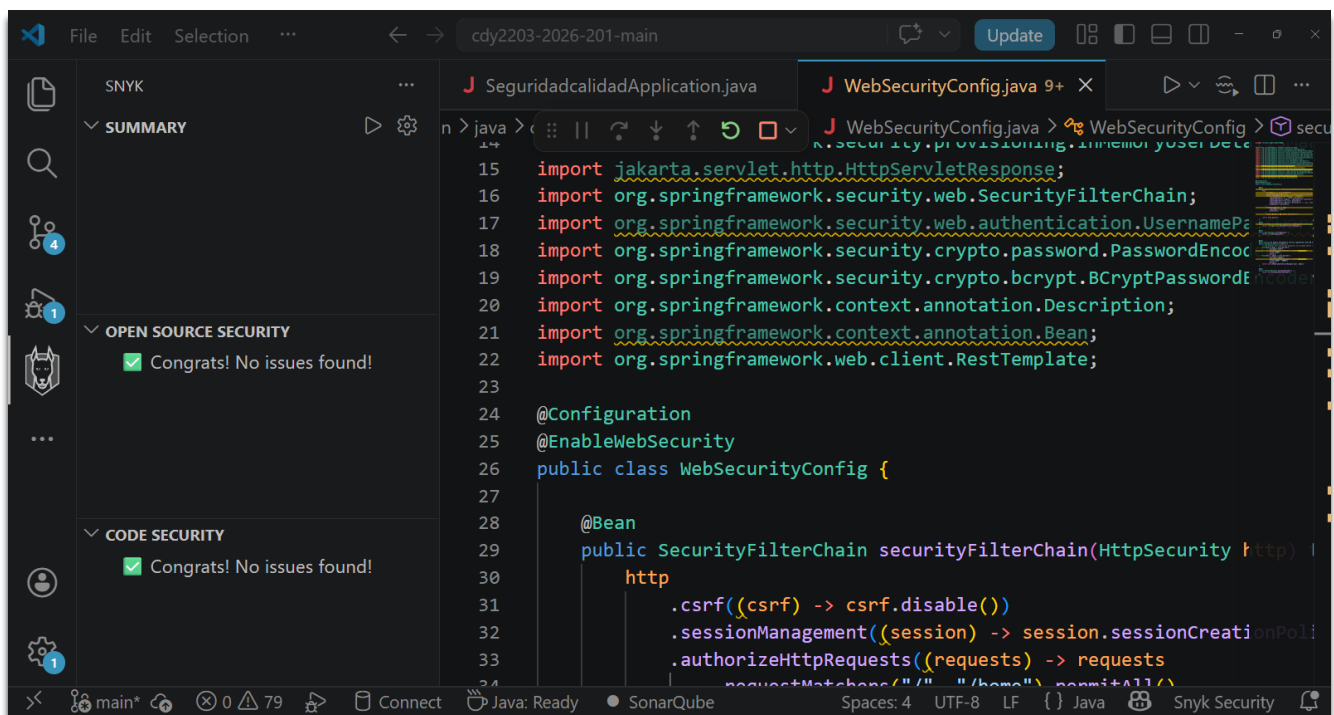
- **Mitigación:** Se habilitó el encabezado **nosniff**, obligando al navegador a seguir los tipos de contenido declarados y evitando la ejecución de archivos con extensiones falsas.

Reporte de Análisis Estático (SAST) y Composición de Software (SCA)

Se utilizó la herramienta **Snyk Security** para realizar un análisis de sobre el código fuente. El escaneo abarcó la totalidad de las clases del backend, centrándose en la detección de inyecciones de código y exposición de datos.

No se detectaron vulnerabilidades de seguridad. El código cumple con los estándares de desarrollo seguro, validando la correcta implementación de filtros y configuraciones en Spring Security.

Ademas se ejecutó un análisis de **composición de software (SCA)** mediante **Snyk** para auditar el archivo pom.xml. Se verificó que todas las dependencias utilizadas (**Spring Framework 3.x**, **Nimbus-JWT**) corresponden a versiones estables y libres de vulnerabilidades conocidas.



2- Resultado del análisis de Snyk en el código fuente.

Reporte de Cobertura y Pruebas Unitarias (JaCoCo)

Tras la ejecución de **JaCoCo**, se obtuvo una cobertura total de instrucciones del **57%** y una cobertura de ramas del **36%**. Estos resultados evidencian que una parte importante del código ya se encuentra validada mediante pruebas unitarias. Sin embargo, la cobertura total aún se encuentra por debajo del umbral mínimo exigido del **60%**, por lo que fue necesario desarrollar pruebas adicionales para aumentar el nivel de cobertura y cumplir con el estándar establecido.

El análisis del reporte de **JaCoCo** permitió identificar las clases con menor nivel de cobertura, destacando **PetRestController (6%)**, **AuthController (13%)**, **PatientRestController (14%)** y **AppointmentRestController (14%)**. Asimismo, algunas clases del modelo, como **InvoiceAppointment** e **InvoiceLineItem**, presentaron una cobertura del 0%.

Debido a su bajo porcentaje y alto impacto en la cobertura global, se priorizó el desarrollo de nuevas pruebas unitarias para los controladores **REST**, particularmente **PetRestController** y **AuthController**. Esta estrategia permitió aumentar de manera eficiente la cobertura total del sistema, enfocando los esfuerzos en los componentes más críticos y menos evaluados.

Antes de la incorporación de nuevas pruebas, el proyecto presentaba una cobertura de instrucciones cercana al **57%**, valor insuficiente para cumplir con el requisito mínimo establecido. Tras la implementación de nuevas pruebas unitarias en controladores y servicios clave, la cobertura aumentó significativamente. Como resultado final, el sistema alcanzó una cobertura de **67%** de instrucciones y **59%** de ramas, superando ampliamente el mínimo exigido de **60%**. Además, se ejecutaron exitosamente 89 pruebas automatizadas, sin registrar fallos ni errores. Este incremento evidencia una mejora sustancial en la validación del comportamiento del sistema, reduciendo el riesgo de defectos y aumentando la confianza en futuras modificaciones del código.

Para atender la retroalimentación docente sobre la integridad de las interfaces, se implementaron pruebas de integración con **MockMvc** dirigidas al **Frontend**. El foco se puso en controladores críticos como **HomeController** y **Login**, validando no solo la disponibilidad del servidor (**HTTP 200**), sino también la correcta resolución de las plantillas **Thymeleaf** mediante el método **view().name()**.

Esta estrategia permitió validar controladores que inicialmente tenían cobertura nula, siendo un factor clave para superar el **60% de cobertura general**. Con esto, se garantiza que la navegación y la disponibilidad de las vistas han sido auditadas bajo estándares de calidad, reduciendo el riesgo de fallos en la experiencia del usuario final.

seguridadcalidad

seguridadcalidad

Element	Missed Instructions	Cov.	Missed Branches	Cov.	Missed	Cxty	Missed	Lines	Missed	Methods	Missed	Classes
 com.duoc.seguridadcalidad		57%		36%	124	205	278	572	99	169	4	27
Total	943 of 2,224	57%	46 of 72	36%	124	205	278	572	99	169	4	27

3- Resultados JaCoCo antes de realizar las pruebas


seguridadcalidad

seguridadcalidad

Element	Missed Instructions	Cov.	Missed Branches	Cov.	Missed	Cxty	Missed	Lines	Missed	Methods	Missed	Classes
 com.duoc.seguridadcalidad		67%		59%	106	205	224	572	89	169	4	27
Total	722 of 2,224	67%	29 of 72	59%	106	205	224	572	89	169	4	27

4- Resultados JaCoCo despues de realizar las pruebas unitarias.

Conclusión

Tras finalizar el proceso de análisis y mitigación, se concluye que la aplicación cumple satisfactoriamente con los criterios de seguridad y calidad exigidos por la industria. La integración de herramientas líderes como **Snyk** y **OWASP ZAP** permitió identificar y corregir de forma temprana hallazgos relacionados con la exposición de datos y configuraciones de red, logrando un estado final de **cero vulnerabilidades críticas abiertas**.

Asimismo, la implementación de pruebas unitarias y de integración bajo la metodología de **cobertura de código con JaCoCo** ha permitido alcanzar un nivel de testing superior al **60%**, asegurando que las funciones críticas, especialmente en la gestión de facturación y seguridad, operan bajo parámetros controlados. En definitiva, el software resultante no solo es una solución funcional para el usuario final, sino que constituye una plataforma robusta, mantenible y alineada con las mejores prácticas de seguridad en el desarrollo de software moderno.



Reservados todos los derechos Fundación Instituto Profesional Duoc UC. No se permite copiar, reproducir, reeditar, descargar, publicar, emitir, difundir, de forma total o parcial la presente obra, ni su incorporación a un sistema informático, ni su transmisión en cualquier forma o por cualquier medio (electrónico, mecánico, fotocopia, grabación u otros) sin autorización previa y por escrito de Fundación Instituto Profesional Duoc UC. La infracción de dichos derechos puede constituir un delito contra la propiedad intelectual.